

Application of Numerical Linear Algebra to PageRank in Search Engine Systems

Artem Okhten

MTH 4311: Numerical Analysis
Professor Jian Du

May 6, 2026

Project Motivation

Main idea: PageRank looks like search engine topic, but mathematically it is numerical linear algebra problem.

- Web pages become nodes in directed graph.
- Hyperlinks become directed edges.
- Ranking vector is found by iterative method.
- Large graph requires sparse matrix computation.

$$r = \alpha S_{\text{full}} r + (1 - \alpha) v$$

This project connects computer systems with eigenvectors, sparse matrices, convergence, and stopping tolerance.

Web Graph Formulation

Each page is one node. If page j links to page i , then

$$A_{ij} = 1$$

For page j with d_j outgoing links,

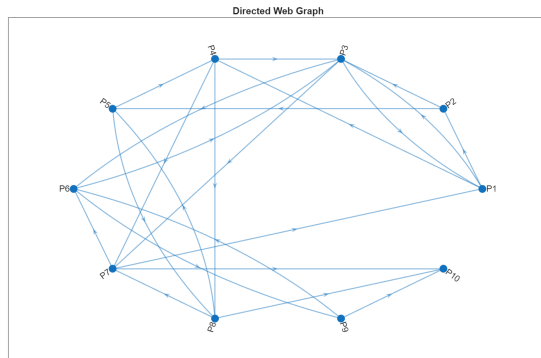
$$S_{ij} = \frac{A_{ij}}{d_j}$$

Interpretation

S is transition matrix. Each non-dangling column describes probability of moving from one page to linked pages.

This is where web ranking becomes matrix problem.

Example Directed Web Graph



The test graph has $n = 10$ pages and 23 directed links. Page 10 is dangling node, meaning it has no outgoing links.

Sparse Transition Matrix

Instead of building transition matrix column by column, I use sparse diagonal scaling:

$$S = AD^{-1}$$

- A : sparse adjacency matrix.
- D : diagonal matrix of outgoing degrees.
- D^{-1} : built in MATLAB using `spdiags`.

Why this matters

MATLAB sparse matrices use compressed sparse column format. Index-vector assembly and diagonal scaling avoid unnecessary memory reallocation.

Dangling Node Correction

A dangling node has no outgoing links. If we fill its whole column with $1/n$, sparse matrix becomes more dense. Instead, I compute dangling mass:

$$m_D^{(k)} = \sum_{j \in \mathcal{D}} r_j^{(k)}$$

Then PageRank update becomes

$$r^{(k+1)} = \alpha S r^{(k)} + \alpha m_D^{(k)} \mathbf{v} + (1 - \alpha) \mathbf{v}$$

Key point

Dangling nodes are handled implicitly, so sparse structure is preserved.

Power Iteration

Starting vector:

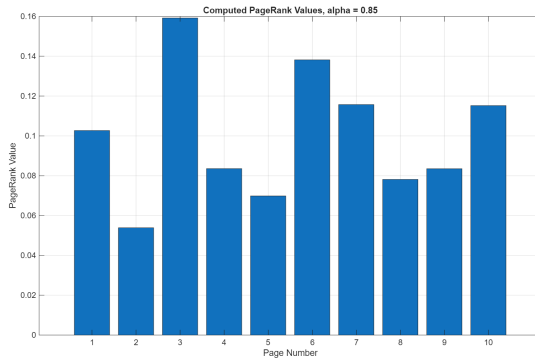
$$r^{(0)} = \frac{1}{n} \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix}$$

Stopping criterion:

$$\|r^{(k+1)} - r^{(k)}\|_1 < 10^{-10}$$

- PageRank vector is dominant eigenvector.
- Perron-Frobenius theorem gives positive dominant eigenvector.
- Power iteration approximates it without matrix inversion.

Computed PageRank Values



Page 3 has highest PageRank value. Page 10 is dangling node, but still has high rank because pages 7, 8, and 9 point to it.

Main PageRank Results

Page	In	Out	Dang.	Rank
3	4	3	No	0.15920
6	3	2	No	0.13819
7	3	3	No	0.11572
10	3	0	Yes	0.11522
1	2	3	No	0.10269

This confirms that the result behaves as probability vector.

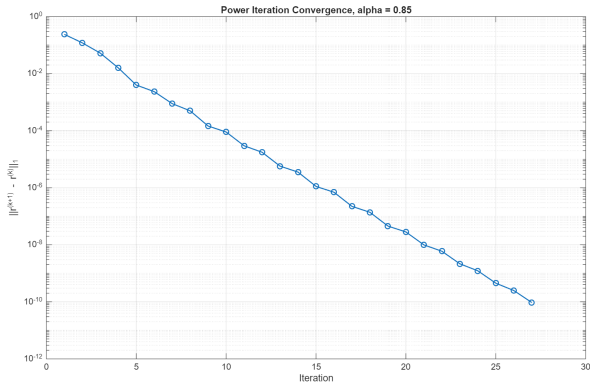
- Power iteration converged in 27 iterations.
- Final L_1 change:

$$9.474 \times 10^{-11}$$

- Sum of PageRank vector:

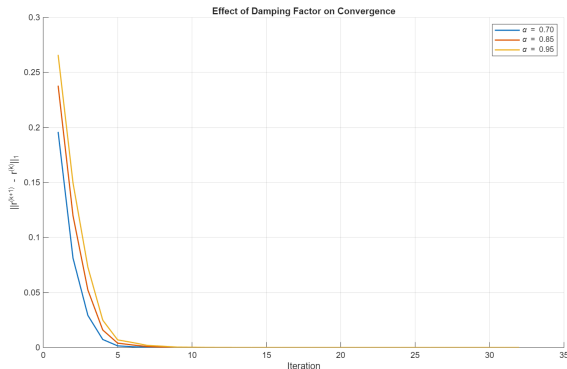
$$1.000000000000$$

Convergence of Power Iteration



Error decreases almost geometrically on semilog scale. This is expected for power iteration when spectral gap is clear.

Effect of Damping Factor



0.70 $\rightarrow$ 22, 0.85 $\rightarrow$ 27, 0.95 $\rightarrow$ 32

As α increases, convergence becomes slower. Numerically, this is connected to smaller spectral gap.

Direct Solve Verification

For small graph only, I also solved:

$$(I - \alpha S_{\text{full}})r = (1 - \alpha)v$$

- Difference between power iteration and direct solve:

$$\|r_{\text{power}} - r_{\text{direct}}\|_1 = 5.539 \times 10^{-11}$$

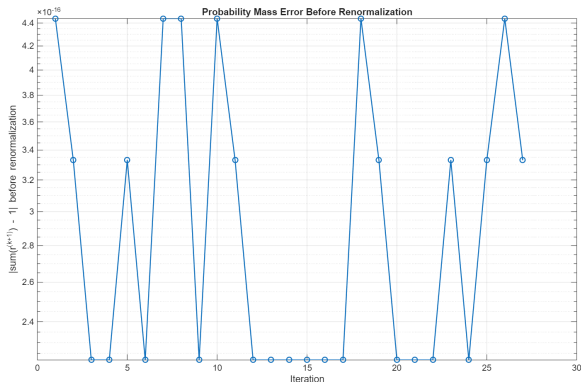
- Direct solve mass error:

$$\left| \sum_i r_{\text{direct},i} - 1 \right| = 0$$

Interpretation

Power iteration solves same fixed-point problem up to numerical tolerance.

Probability Mass Check



$$\max \left| \sum_i r_i^{(k+1)} - 1 \right| = 2.220 \times 10^{-16}$$

This is machine precision level. It shows probability mass is preserved correctly, and renormalization only corrects round-off drift.

Numerical Challenges

- **Slow convergence:** controlled by spectral gap.
- **Dangling nodes:** cannot fill dense columns in sparse matrix.
- **Large storage:** dense $n \times n$ matrix is impossible for web-scale graph.
- **Stopping tolerance:** accuracy and runtime must be balanced.
- **Round-off drift:** probability mass should stay close to 1.

Main numerical lesson

Algorithm must match both mathematical model and memory structure.

Conclusion

- PageRank starts as search ranking problem.
- Mathematically, it becomes eigenvector and fixed-point problem.
- Sparse power iteration gives practical numerical solution.
- $S = AD^{-1}$ and implicit dangling-node correction preserve sparsity.
- Direct solve check confirms correctness for small graph.
- Damping factor affects both model and convergence speed.

Computer systems problem $\Rightarrow$ Numerical linear algebra problem